

Sokin: Senior QA Analyst (Mobile) Assignment

Candidate: Paulo Goncalves Junior

Email: paulorrg_junior@hotmail.com

Mobile quality-engineering exercise for `automationexercise.com`, treated as the functional reference for a hypothetical native **iOS/Android** e-commerce app.

This repository contains all five deliverables: a test strategy, an exploratory testing report, answers to the product-discovery questions, a working automation framework, and the "how I'd automate this" write-up.

My approach in one minute

1. **Prioritise the buying journey.** For an e-commerce app, value lives in account access, browsing, cart, and checkout. I test and automate that first, and deliberately leave broader testing (many devices, visual checks, other languages, strict performance targets) for later.
 2. **Automate the repeatable, explore the rest.** Stable, repeatable flows are automated, while the device, network, and human dependent parts are checked by hand through exploratory testing.
 3. **Write tests once, run everywhere.** The framework uses Page Objects over a **cross-platform selector map** (`web | android | ios`), so the same specs target the responsive web today and native iOS/Android when the app build exists.
 4. **Prove it runs.** Since there is no real native app (and no emulators are assumed), the framework is genuinely executable today against the responsive site under **mobile-emulated Chrome**. `npm run test:web` runs green with only Node and Chrome. The Appium/iOS/Android targets are fully wired and ready for app artifacts and SDKs.
-

Deliverables map

Deliverable	Where to find it
1. Mobile Test Strategy	The "Mobile Test Strategy" section of this document
2. Exploratory Testing Report	The "Exploratory Testing Report" section of this document
3. Mobile Automation (code and framework)	The project's automation code (the <code>config/</code> and <code>test/</code> folders)
4. How I Would Automate This	The "How I Would Automate This Mobile App" section of this document
5. Product Discovery Questions	The "Product Discovery Questions" section of this document
Evidence (screenshots)	Embedded throughout, within the relevant sections

The automation framework

```
config/
  wdio.shared.conf.ts      # platform-agnostic base (framework, timeouts, isolation hook
  wdio.web.conf.ts         # RUNNABLE TODAY: mobile-emulated Chrome (Node + Chrome only)
  wdio.android.conf.ts     # Appium + UiAutomator2 (needs Android SDK + .apk)
  wdio.ios.conf.ts         # Appium + XCUITest (needs Xcode + .app/.ipa)
test/
  specs/                   # business flows + assertions only (no selectors)
    auth.e2e.ts            #   register a new user end-to-end; reject invalid login
    cart.e2e.ts            #   add multiple products; cart persists across refresh
  pageobjects/             # Page Object Model: actions/intent per screen
  selectors/index.ts       # ONE cross-platform locator map { web, android, ios }
  data/users.ts            # unique-per-run test-data factory
  support/platform.ts      # resolves the active platform
.github/workflows/ci.yml   # PR gate (web-emulation) + scaffolded native jobs
```

Design highlights (full reasoning in the "How I Would Automate This Mobile App" section of this document):

- **Page Object Model:** specs read as intent (`AuthPage.login(...)`), and selectors never leak into tests.
- **Cross-platform selectors:** each element is `{ web, android, ios }`, and native defaults to accessibility ids (stable and accessibility friendly). Only this map changes when the real app lands.
- **Resilient interactions:** a shared `click` waits, centers, and falls back to a DOM click when overlays intercept taps.
- **Test data and isolation:** a unique user per run, and cookies cleared before each web test so order never matters.

Run it

Prerequisites: Node 20+ and Google Chrome. (WebdriverIO v9 auto-manages the matching driver.)

```
npm install
npm run typecheck      # TypeScript: zero errors
npm run test:web       # runnable proof: mobile-emulated Chrome vs. the live site
npm run test:web:headed # same, with a visible browser
```

Expected result (verified on this machine):

```
auth.e2e.ts    ✓ registers a brand-new user end-to-end and signs them in
               ✓ rejects invalid credentials with a clear, user-facing error
cart.e2e.ts    ✓ keeps all added items after a page refresh
Spec Files:    2 passed, 2 total
```

Native targets (framework ready; require the app build + SDKs):

```
ANDROID_APP=/abs/path/app.apk npm run test:android    # Appium 2 + uiautomator2 + emulator
IOS_APP=/abs/path/App.app      npm run test:ios        # Appium 2 + xcuitest + simulator
```

Notes & assumptions

- No real native app exists for this exercise, so "mobile automation" is implemented as a native-shaped framework with a runnable web-emulation proof. The same specs and Page Objects drive all three targets.
- The registration test creates real accounts on the public practice site, using a **unique random email per run** to stay idempotent.
- Environment here: Node v20.11, macOS; no Appium/emulators installed, which is why the web target is the runnable proof.

Mobile Test Strategy

This is how I would approach testing the mobile version of this app. The product I am looking at is the website `automationexercise.com`, and for this exercise I am treating it as the reference for a native iOS and Android app that does the same things.

My goal in this document is not to list every test I could possibly run. It is to show what I would focus on first, what I would leave for later, and the reasoning behind those choices. I think that prioritisation is the real skill in QA, so that is what I want to make clear.

What I am trying to do here

When I plan testing for an app like this, I start by asking a simple question: where does this product make or lose money and trust? For an online shop, that is a fairly small set of things. If those break, nothing else matters. So I put most of my early effort there, and I am deliberate about the parts I choose to leave until later.

The user flows I would test first

I prioritise flows by two things together: how important they are to the business, and how likely they are to break specifically on a phone. Here are the five I would start with, and why.

1. **Creating an account and logging in.** This is the front door. If sign up or login does not work, the user never reaches the cart or the checkout, so everything else is blocked. It also has a lot of input fields (email, password, personal details), and it is the screen most affected by mobile keyboards, autofill, and staying logged in between sessions.
2. **Finding products (browsing, search, and filters).** This is the top of the journey. If people cannot find what they want, they cannot buy it. On a phone this is very touch heavy (lots of scrolling and tapping), images need to load nicely even on a weak connection, and it is usually the first place performance problems show up.
3. **Managing the cart (adding, updating, removing, and keeping items).** This is the bridge between looking and buying. The part I care about most on mobile is whether the cart survives. People get interrupted constantly on phones, so I want to know the cart is still there when they come back.
4. **Checkout and payment.** This is the single most important flow for revenue. A problem here is a direct, measurable loss, so it gets the deepest testing even though it gets less traffic than browsing. I would test both the happy path and a lot of "what if the user does something wrong" cases.
5. **Staying logged in and keeping state.** This runs across everything. It covers things like staying signed in, the cart surviving after the app is closed, links opening the right screen, and the back

button behaving as expected. This is the area where a mobile app most often behaves differently from the website.

If I could only pick three of these, I would keep account access, cart, and checkout, because that is the path that actually leads to a sale.

What I would automate and what I would test by hand

The way I split this is simple. I automate the things that are stable and that I will need to check over and over again. I test by hand the things that need human judgement or that depend on a real device.

Things I would automate (my safety net):

- The main account flows: registering, logging in, logging in with the wrong password, and trying to register an email that already exists. These have clear right and wrong answers, so a machine can check them on every build. I have already built these in this project.
- Adding products to the cart and checking that the cart keeps its contents after a refresh. This is also already built.
- A basic checkout check that reaches the payment step with a known product, stopping before any real payment.
- Running the same checks on both iOS and Android, which is cheap to do once the framework is set up and catches differences between the two platforms.

I automate these because they run constantly, they have a clear pass or fail, and re-checking them by hand on every release across two platforms would be slow and error prone.

Things I would test by hand (where judgement matters):

- First time use and permission prompts (notifications, location), including what happens when the user says no.
- Real gestures and how the app feels in the hand: swiping, long pressing, pull to refresh, reaching things one handed, larger font sizes, and dark mode.
- Interruptions and bad connections: a phone call mid checkout, the app being sent to the background, airplane mode, a weak signal, and recovering when the connection comes back.
- Visual polish and accessibility: how the layout looks on different screen shapes, and how it works with screen readers like VoiceOver and TalkBack.

I keep these manual because they are hard to judge automatically, they change a lot, and a person can evaluate them faster and more sensibly than a brittle script could.

The biggest risks that are specific to mobile

These are the things that worry me most about this product running as a real app, rather than as a website.

1. **The app being closed and reopened by the phone.** This is my number one concern, so it is worth explaining in plain terms. On a phone, people switch apps all the time. Someone might add

an item to the cart, then jump to WhatsApp to ask a friend, take a call, or just lock the screen. While the app is sitting in the background, the phone's operating system is allowed to shut it down on its own to free up memory. This happens a lot, especially on cheaper Android phones. When the user comes back, the app has to rebuild itself from scratch. The question I would test is: does the cart still have the items, and is the user still logged in? On a website this almost never happens, because the page just stays open in the browser. On a phone it happens constantly, and that is exactly why something can work perfectly on the web but still break in the app.

2. **Bad or changing connections.** Phones move in and out of signal. The app needs to handle a slow connection, a request that drops halfway, and being fully offline, without confusing the user or, worst of all, charging them twice at checkout.
3. **Different devices and screen sizes.** There are a huge number of Android phones with different sizes, operating system versions, and manufacturer tweaks, plus iPhones with notches and safe areas. Buttons and layouts have to hold up across all of them.
4. **Payment safety.** I want to be sure the user cannot accidentally submit a payment twice on a flaky connection, that switching to a banking app and back works correctly, and that no duplicate orders or charges are created.
5. **Performance on cheaper phones.** Long, image heavy product lists can become slow and jerky on low end devices, and can use a lot of memory and battery. I would keep an eye on scrolling smoothness and how long the app takes to start.
6. **Permissions and links.** Push notifications, and links that should open the right screen inside the app, both need to behave correctly.
7. **Keeping user data safe on the device.** Login tokens and personal details should be handled carefully, and the right thing should happen when the user logs out or loses their phone.

What I would not test at the start

Choosing not to test something is a decision, not an oversight, so I want to be open about it. Early on, I would not spend time on:

- **Testing on every possible device.** I would start with two or three that represent most users (one cheaper Android, one modern Android, one current iPhone) and grow from there. Trying to cover every device from day one is not worth the effort.
- **Pixel perfect visual checks.** Useful later, but early on they create a lot of noise and maintenance for little value.
- **Different languages, currencies, and right to left layouts.** Only worth it if and when the product targets those markets.
- **Formal performance benchmarks as a hard gate.** I would still look at performance by hand early, but strict targets come after the main tests are trustworthy.
- **Rarely used features**, such as minor account settings or secondary pages, until the core buying journey is solid.

- **Checking that ads and analytics are correct**, beyond making sure they do not get in the way of the user. (In fact, the ad scripts on the reference site actively got in the way of automation, so I treat them as noise to work around rather than something to validate.)

The thread running through all of this is the same: protect the buying journey first, then widen the coverage once I am confident the core is solid.

In short

I would test the buying journey first, meaning account access, browsing, cart, and checkout, before anything else. I would automate the repeatable checks and explore the device and human dependent parts by hand. I would treat the app being closed and reopened, bad connections, device variety, and payment safety as the defining mobile risks. And I would hold off on breadth, like full device coverage, visual checks, extra languages, and strict performance targets, until the core is solid, because knowing what to leave for later is part of doing this well.

Exploratory Testing Report

About this session

Field	Value
Tester	Paulo
Date	2026-06-08
Duration	About 45 minutes of active testing
Environment	A real iPhone (iOS Safari) plus desktop Chrome with phone sized emulation (iPhone 12 Pro and Pixel 7)
Site tested	https://automationexercise.com
Evidence	Screenshots, plus console and network captures from the browser tools (in ../evidence/)

What I set out to explore (my charter)

I wanted to walk through the core shopping journey the way a real person on a phone would: creating an account and logging in, browsing and searching for products, and then using the cart and checkout. My aim was to find broken behaviour, gaps in validation, places where the experience is confusing, and risks that are specific to using this on a phone.

What I looked at: sign up and login (including doing things wrong on purpose), browsing, searching and filtering, adding, updating and removing items in the cart, whether the cart survives a refresh, checkout and payment, and how errors are handled.

What I left out of this session on purpose: a full accessibility review, performance measurement, testing across many devices, and completing a real payment.

The short version

The core journey works well enough to get from browsing to a confirmed order, but along the way I found **21 issues**. One of them is serious enough that I would call it a blocker: the payment screen accepts clearly fake details and still confirms the order. Six more are high priority, covering things like missing validation, a cart you cannot really manage, dangerous one tap account deletion, and a mobile specific problem with adding items to the cart. The complete list, with steps and suggested fixes for each, is in the "Issues and Improvements Tracker" section of this document. The most important ones are written out below with evidence.

Severity	Count
Critical	1

Severity	Count
High	6
Medium	9
Low	5

Things I noticed along the way

- The app barely validates anything on its own. The email checks come from the browser, not the app, and on the registration and payment screens almost any text is accepted.
- Money is shown without proper formatting anywhere it appears, for example Rs. 1500 , or a line total written as 61727722500 . That makes prices hard to read and trust, which matters even more on a global mobile app.
- The phone experience behaves differently from the desktop one. Adding to the cart needs a second tap, and the confirmation popup appears in the wrong place. These are the classic signs of desktop hover behaviour being reused on a touch screen.
- Actions that cannot be undone have no safety net. Both deleting your account and removing a cart item happen instantly with no confirmation.
- The site itself is fragile under load. A "queue full" page with server errors appeared during my session. I noted it, but I treat it as the demo site's own limitation rather than a product problem.

The main issues I found

The full list is in the tracker. Here are the ones that matter most, with the evidence.

1. Payment accepts fake details and still confirms the order (PAY-01). Severity: Critical

What happens: on the payment screen I can type something obviously wrong, like the number 1 in every field, and the order is still confirmed. There is no real check on the card number, expiry, or security code. The only thing stopping me is the browser's built in "this field is required" message on an empty field.

How to reproduce:

1. Add a product and go through checkout to the payment screen.
2. Type 1 into Name on Card, Card Number, CVC, and Expiration.
3. Tap Pay and Confirm Order.

What I expected: the payment details to be properly checked and rejected when they are clearly invalid.

What actually happened: the order was confirmed with nonsense payment details.

Why this matters: this is the step where money changes hands, so it is the most important thing to get right. In a real product this would be unacceptable, and on a phone it is made worse by shaky

connections that can cause double submissions. This is the first thing I would want fixed.

2. The signup form accepts a badly formed email (AUTH-01). Severity: High

What happens: the email field only uses the browser's basic check. So `missing@` style mistakes get caught by the browser, but `paulo@test` (which has no proper domain) is accepted and lets me create an account. In other words, the app adds no email checking of its own beyond the bare minimum the browser provides.

How to reproduce:

1. Go to Signup and Login, then New User Signup.
2. Enter the name `Paulo` and the email `paulo@test`, then tap Signup.

What I expected: an email without a real domain to be rejected, and ideally confirmed by sending a verification email.

What actually happened: it was accepted and moved on to the account information page.

Why this matters: the email address is how the account is identified and how the customer receives order confirmations and password resets. Accepting unreachable addresses leads to junk accounts and broken communication.

Evidence, showing the contrast between what the browser blocks and what the app lets through:

h to your account

Email Address

Password

Login

OR

New User Signup!

Paulo

paulo@test.com

Email Address

Signup

Please include an '@' in the email address. 'paulo@test.com' is missing an '@'.

SUBSCRIPTION

Your email address

Treino de IA de Código

Junte-se a 100 mil+ especialistas a ganhar 50-60\$+/h a treinar modelos de IA

dataannotation.tech

OPEN

word

Login

OR

New User Signup!

Paulo

paulo.test.com@

Signup

Please enter a part following '@'. 'paulo.test.com@' is incomplete.

SUBSCRIPTION

Your email address

Treino de IA de Código

Junte-se a 100 mil+ especialistas a ganhar 50-60\$+/h a treinar modelos de IA

dataannotation.tech

OPEN

ENTER ACCOUNT INFORMATION

Title

☐ Mr. ☐ Mrs.

Name *

Paulo

Email *

paulo@test

Password *

Date of Birth

Day

Month

Year

☐ Sign up for our newsletter!

☐ Receive special offers from our partners!

ADDRESS INFORMATION

First name *

Last name *

3. Adding to the cart needs two taps on mobile (ATC-02). Severity: High (mobile)

What happens: on a phone, the first tap on a product's Add to Cart button does not add it. Instead an orange slider appears, and I have to tap Add to Cart a second time before the product is actually added.

How to reproduce:

1. On a phone, open Products.
2. Tap Add to Cart on a product. An orange overlay appears.
3. Tap Add to Cart again. Now it is added.

What I expected: one tap to add the product.

What actually happened: it took two taps, because a desktop hover effect is being shown on a touch screen.

Why this matters: every extra tap on the path to buying costs sales, and this happens right on the most important action, specifically on mobile. This is exactly the kind of phone only problem a mobile QA role should catch.

Evidence: seen on the device. The related confirmation popup is captured in `../evidence/atc03-added-popup-mobile.png`.

4. You cannot change the quantity in the cart (CART-01). Severity: High

What happens: once items are in the cart, there is no way to change the quantity on the cart screen. The quantity can only be set on the product page before adding.

How to reproduce: add a product, open the cart, and try to change its quantity.

What I expected: to be able to edit the quantity in the cart, with the total updating.

What actually happened: the quantity is fixed.

Why this matters: changing quantity in the cart is something every shopper expects. Without it, people have to remove and re-add items, which is annoying and can lose the sale.

Evidence: `../evidence/cart04-line-total-and-qty.png` (the quantity appears as a fixed value).

5. "Delete Account" happens with no confirmation (AUTH-04). Severity: High

What happens: a logged in user can permanently delete their account with a single click. There is no confirmation step and no need to re-enter the password.

How to reproduce: log in, then click Delete Account.

What I expected: a confirmation step before something this final.

What actually happened: the account was deleted right away.

Why this matters: this is permanent and cannot be undone. An accidental tap, which is easy to do on a phone, would wipe the account with no way back.

6. On mobile, the duplicate email error scrolls out of view (AUTH-03, also answers Discovery Q1). Severity: Medium (mobile)

What happens: when I try to sign up with an email that already exists, the app correctly shows the message "Email Address already exist!". But on mobile the page jumps back to the top, so the message, which sits lower down, is off screen. The user has to scroll down to find out why the signup did not work.

How to reproduce:

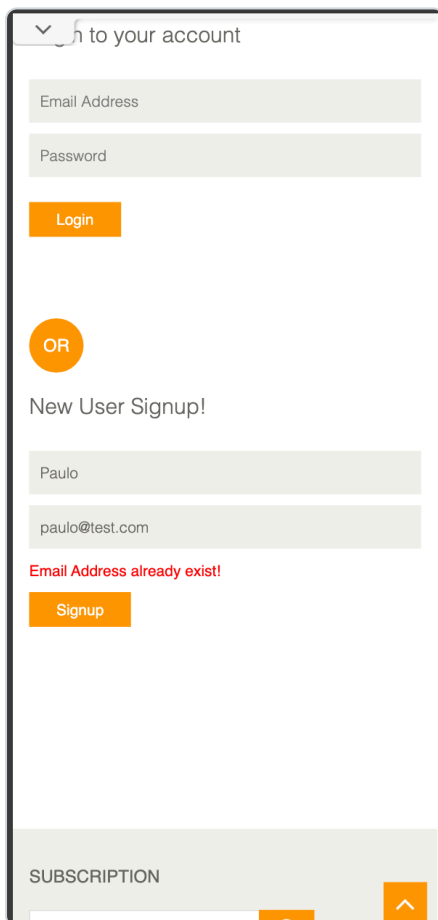
1. On a phone, go to New User Signup.
2. Enter an email that already exists (paulo@test.com) and tap Signup.

What I expected: the error to be visible, either by scrolling to it or showing it next to the field.

What actually happened: the page jumped to the top and the red error was left below the fold.

Why this matters: on a small screen, an error the user cannot see feels like the app simply did nothing. People may give up or keep retrying without understanding why. This is a clear mobile experience problem.

Evidence: [../evidence/q1-duplicate-email-mobile.png](#) .



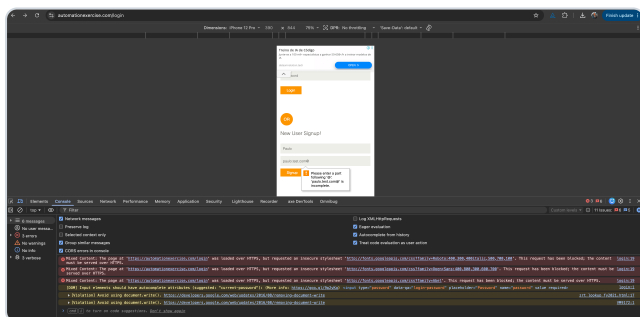
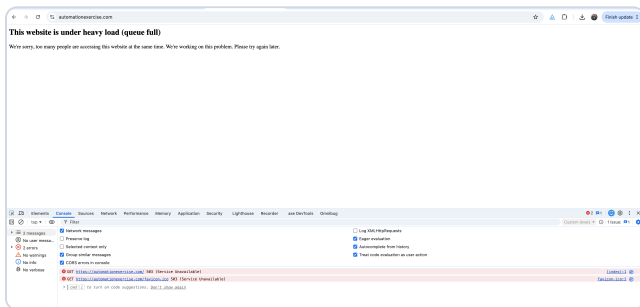
The remaining issues (no overall cart total, unlimited quantity, a wrong product showing under a category filter, prices and totals without currency formatting, the Enter key not running a search, the popup behaviour, no field highlight on empty required fields, removing a cart item with no confirmation,

the misaligned search button, the site overload page, and the lack of a profile area) are all written up with steps, severity, and suggested fixes in the "Issues and Improvements Tracker" section of this document.

What I saw in the browser tools (console and network)

I also kept an eye on the browser's developer tools, which gives some useful extra evidence:

- The site returned **503 Service Unavailable** errors on the home page and favicon when it hit its "heavy load (queue full)" state. The console and network panels both show this (`stab01-console-503.png` , `stab01-network-503.png` , `stab01-heavy-load-mobile.png`).
- On the login page there were **mixed content** warnings: the page is served securely over HTTPS, but it tries to load fonts over insecure HTTP, so the browser blocks them (`console-login-mixed-content.png`).
- The password field is missing the `autocomplete` setting that browsers recommend, and there were warnings about an outdated way of writing to the page (`document.write`).



What worked as expected

- Registering a brand new user works and logs you in.
- Trying to register an email that already exists is correctly blocked with a clear message. The only problem is where that message ends up on mobile (issue AUTH-03).
- Logging in with the wrong password is rejected with a clear error. I confirmed this by hand and with automation.
- Adding to the cart and keeping the items after a refresh works. I confirmed this with automation (see Discovery Q2).
- The browser correctly blocks the most obvious bad emails, such as a missing or trailing `@` .

Evidence index

Reference	File	What it shows
AUTH-01	evidence/auth01-email-missing-at.png	Browser blocks a missing @
AUTH-01	evidence/auth01-email-trailing-at.png	Browser blocks a trailing @
AUTH-01	evidence/auth01-email-no-tld-accepted.png	paulo@test is accepted and moves to account info
AUTH-03 / Q1	evidence/q1-duplicate-email-mobile.png	The "Email Address already exist!" message on mobile
ATC-03	evidence/atc03-added-popup-mobile.png	Where the "Added" popup appears on mobile
CART-04 / CART-06	evidence/cart04-line-total-and-qty.png	An unformatted line total (61727722500) and a quantity of 123455445
BRW-01	evidence/BRW-01.png	Kids and Dress filter showing a product that is not a dress
BRW-02	evidence/BRW-02.png	Prices shown as Rs. 500/400/1000/1500 with no formatting
BRW-03	evidence/BRW-03.png	"Shirt" typed in search, but pressing Enter does nothing
BRW-04	evidence/BRW-04.png	The search button not lined up with the search bar
STAB-01	evidence/stab01-heavy-load-mobile.png	The "under heavy load (queue full)" page
STAB-01	evidence/stab01-network-503.png	Network panel showing the 503
STAB-01	evidence/stab01-console-503.png	Console showing 503 Service Unavailable
Console	evidence/console-login-mixed-content.png	Mixed content, autocomplete, and document.write warnings

Notes and limits of this session

- I focused on the buying journey within my time limit, and deliberately left aside the wider areas like full device coverage, accessibility, and performance.
- I did not complete a real payment. I observed the payment acceptance problem (PAY-01) up to the point where the order is confirmed.
- I confirmed cart persistence after refresh (Discovery Q2) using the automated test rather than a manual screenshot.
- I treat the site's overload page (STAB-01) as a limitation of the demo environment rather than the product, but the way it is shown to users and the errors behind it are still worth noting.

- If I had more time, I would focus next on what happens when the app is closed and reopened on a real phone, recovering from going offline and back online, and a proper accessibility pass with VoiceOver and TalkBack.

Product Discovery Questions

These are my answers to the three product questions. For each one I describe what I actually saw and how I checked it. Where an issue has a tracking ID, it is written up in full in the "Issues and Improvements Tracker" section of this document. The testing was done on 2026-06-08, using a real iPhone (iOS Safari) and desktop Chrome with phone sized emulation.

Question 1. Creating an account with an email that already exists

What the system does: The app handles this correctly. When I fill in the New User Signup form with an email that is already registered (paulo@test.com), it does not create a second account. It stays on the signup form, shows an error, and does not move on to the account information step.

The error message: A red message saying "Email Address already exist!" appears under the email field.

Does the behaviour seem correct? Mostly yes, but with two real problems around it.

First, blocking the duplicate is the right thing to do, and the message itself is clear. So the core behaviour is correct.

However, on mobile there is a problem with the message being seen at all. When I submit, the page jumps back to the top, and the error sits lower down, off screen. The form looks like it just reset itself for no reason, and the user has to scroll down to find out why. The message should either scroll into view or appear right next to the field. I tracked this as issue AUTH-03.

Second, there is a gap in how emails are checked in the first place. The signup field only uses the browser's basic email check. The browser blocks obvious mistakes like a missing @ or a trailing @ , but paulo@test , which has no real domain, is accepted and lets me create an account. So while duplicate emails are caught, badly formed emails are not, because the app does no checking of its own beyond the browser's minimum. I tracked this as issue AUTH-01. (One small extra point: telling the user plainly that an email "already exists" also makes it easier for someone to find out which emails are registered. That is fine for a demo, but worth thinking about for a real product.)

How I checked it: I reproduced the duplicate email block on mobile, shown in evidence/q1-duplicate-email-mobile.png . I also tested several email formats: missing @ and a trailing @ were both blocked by the browser (auth01-email-missing-at.png , auth01-email-trailing-at.png), while paulo@test was accepted and moved on to the account information page (auth01-email-no-tld-accepted.png).

h to your account

Email Address

Password

Login

OR

New User Signup!

Paulo

paulo@test.com

Email Address already exist!

Signup

SUBSCRIPTION

Treino de IA de Código

Junte-se a 100 mil+ especialistas a ganhar 50-60\$/h a treinar modelos de IA

dataannotation.tech

OPEN >

ENTER ACCOUNT INFORMATION

Title

☐ Mr. ☐ Mrs.

Name *

Paulo

Email *

paulo@test

Password *

Date of Birth

Day Month Year

☐ Sign up for our newsletter!

☐ Receive special offers from our partners!

ADDRESS INFORMATION

First name *

Last name *

Question 2. Adding several products to the cart and refreshing the page

What happens: The cart keeps its contents after a refresh. The items and quantities are still there when the page reloads.

Is this expected? Yes. This is what users expect and want. A cart that survives a reload protects the path to buying, and it makes sense that the site remembers the cart through a stored session rather than only keeping it in the current page.

What risk could this introduce on a mobile app? Keeping the cart is good, but it raises some questions that are specific to a phone, and these are exactly the things I would want to test:

- Does the cart survive the app being fully closed and reopened, not just a page refresh? On a phone the operating system can shut the app down in the background, which is much harsher than a simple reload, and that is where mobile apps often behave differently from the website.
- Is the cart tied to the user's account and shared across their devices, or is it only stored on that one phone and lost if they reinstall or log out?
- If a cart is kept for a long time, the items in it might go out of stock or change price by the time the user checks out. So keeping the cart needs to be balanced against showing up to date stock and prices.

How I checked it: This is backed by an automated test included with this submission (the cart test). The test adds two products, opens the cart, checks that there are two items, refreshes the page, and checks again that there are still two items. It passes. I chose to make this an automated check on purpose, so that if this behaviour ever breaks in the future, it gets caught straight away. My manual check matched the automated result. I did not take a separate manual screenshot of the refresh, since the behaviour is already covered by the test.

Question 3. A browsing behaviour that could confuse or frustrate users

My main example: prices are shown without any currency or number formatting (issue BRW-02).

What I saw: On the Home and Products screens, prices appear as plain values like Rs. 500 , Rs. 1000 , and Rs. 1500 , with no clear currency and no proper number formatting. In the cart it gets worse, where a line total shows up as Rs. 61727722500 , just a long string of digits (issue CART-04).

Why it could be a problem:

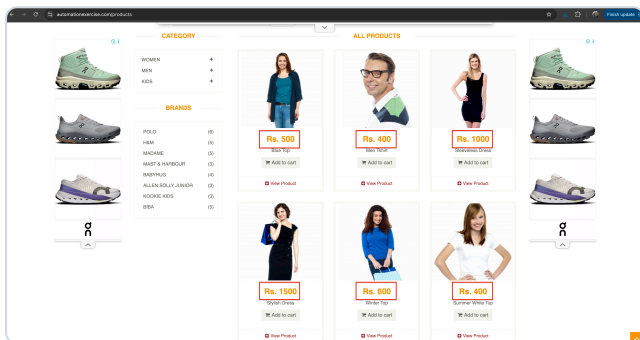
- It is unclear. "Rs." (Indian Rupees) will not be understood by a lot of people around the world, and a number like 1500 with no separators or decimals is easy to misread.
- It hurts trust at the exact moment someone is about to spend money, which is the worst place to create doubt.
- It is worse on a phone. There is less space, people read at a glance, and they are often shopping while out and about. A huge unformatted number like the cart total is genuinely hard to read on a

small screen.

How I would improve it:

- Format every price properly, with the right currency symbol and correct grouping and decimals, for example ₹1,500.00 .
- Base the currency and format on the user's region, so it makes sense in each market.
- Add an automated check that prices match the expected format, so this does not quietly break again later.

How I checked it: I captured the products list with the plain prices (evidence/BRW-02.png) and the unformatted cart total (evidence/cart04-line-total-and-qty.png).



A couple of other strong candidates I also found, in case they are of interest (both are in the tracker):

- The Kids and Dress category filter shows a product that is not a dress ("Sleeves Top and Short"), which makes the filtering feel unreliable (evidence/BRW-01.png).
- On desktop, pressing Enter in the search box does nothing. Only clicking the search button works (evidence/BRW-03.png).

Evidence index

Question	File	What it shows
Q1	evidence/q1-duplicate-email-mobile.png	"Email Address already exist!" on mobile
Q1	evidence/auth01-email-no-tld-accepted.png	paulo@test accepted and moving to account info
Q1	evidence/auth01-email-missing-at.png , auth01-email-trailing-at.png	The browser blocks these two
Q2	Automated cart test (passing)	The cart survives a refresh
Q3	evidence/BRW-02.png	Prices without currency formatting
Q3	evidence/cart04-line-total-and-qty.png	An unformatted cart line total
Q3	evidence/BRW-01.png	A wrong result under the category filter

How I Would Automate This Mobile App

In this section I want to explain, in plain terms, how I would build the automation for this app and how it would grow over time. The framework in this project is a small, working version of exactly what I describe here, so it is not just theory.

Which tools I would choose, and why

My main choice would be **WebdriverIO together with Appium, written in TypeScript**.

Appium is the industry standard for mobile testing. The big advantage is that one set of tests can drive both iOS and Android, instead of writing and maintaining two separate test suites. That matters a lot when the same product ships on both platforms.

WebdriverIO is the tool that actually runs the tests and ties everything together. It works very naturally with TypeScript and with Appium, it can run tests in parallel, retry flaky ones, connect to device clouds, and produce good reports. It is also widely used in the industry, which means it is easy to hire people who already know it.

I chose TypeScript because it catches a lot of small mistakes while I am writing the tests, before they ever run. As the test suite grows, that safety makes the whole thing much easier to maintain and change.

A quick word on the native tools (Apple's XCUITest and Google's Espresso). They are excellent, and I would still use them, but for a different job. They are best for fast, low level tests written by the app developers themselves. For the end to end user journeys that QA owns, I prefer Appium and WebdriverIO because one suite covers both platforms. So rather than choosing one over the other, I would use them together, each for what it does best.

How I would structure the framework

The main idea is that a test should be written once and then run on web, iOS, and Android, with only the underlying details changing per platform. Here is how the project is laid out:

```
config/      wdio.shared.conf.ts    # the settings shared by every platform
              wdio.web.conf.ts      # runnable today: mobile-emulated Chrome
              wdio.android.conf.ts   # Appium for Android
              wdio.ios.conf.ts       # Appium for iOS

test/
  specs/      *.e2e.ts      # the actual tests: business steps and checks
  pageobjects/ *.page.ts    # one file per screen, holding the actions
  selectors/  index.ts      # one map of how to find each element per platform
  data/       users.ts      # generates fresh test data for each run
  support/    platform.ts   # knows which platform is running
```

The decisions behind this:

- **Page Objects.** Each screen has its own file that describes what you can do on it, such as "log in" or "add to cart". The tests themselves just say what the user does, in plain steps. This means that if a screen changes, I only fix it in one place.
- **One place for element locations.** Every element knows how to be found on web, Android, and iOS. For the native apps I rely on accessibility identifiers, which are the most stable way to find things and also help users who rely on screen readers. When the real app is built, this is the only file that needs to change.
- **Tests that do not give up easily.** There is a shared helper that waits for an element, scrolls to it, and tries an alternative way to tap it if something gets in the way. On the public site, ads literally sat on top of buttons and blocked them, so this kept the tests reliable.
- **Fresh data every time.** Each run creates a brand new user, so the registration tests never clash with an account that already exists.
- **Tests that do not depend on each other.** State is cleared before each test, so the order they run in never matters.

This keeps things small but ready to grow. Adding a new screen means one new Page Object and one entry in the locator map. Adding a new platform means one new config file.

How the tests would run on iOS and Android

- **On my own machine while developing,** I would use Appium with simulators (iOS) and emulators (Android). The framework is already set up for this; it just needs the Apple and Android development tools installed, plus a built version of the app to point it at.
- **The app itself** comes from the app team's own build process as a file (an `.apk` for Android, an `.app` or `.ipa` for iOS). The automation never builds the app, it just runs against it.
- **For wide device coverage,** I would use a device cloud such as BrowserStack, Sauce Labs, or Firebase Test Lab. These let me run on many real phones and operating system versions without having to own a room full of devices. Switching to them does not require changing the tests.
- **The web version** (`npm run test:web`) runs the same tests against the responsive website in a mobile sized Chrome window. It only needs Node and Chrome, runs in seconds, and is the part that runs successfully in this project today. It is a fast way to prove the tests and Page Objects work without needing any phone or emulator.

How this fits with CI/CD

I would set this up so the cheap, fast checks run all the time, and the heavier, broader checks run on a schedule.

- **On every code change (pull request):** run the type check and the fast web tests. This takes minutes, needs no devices, and stops broken changes from being merged.
- **After merging, and overnight:** run the full Appium tests on a device cloud, across a sensible set of iOS and Android devices, once the latest app build is ready.
- **Before a release:** run a wider set of devices plus the full checkout journey, as a final gate.

- **Reporting:** every run produces a clear report, with screenshots and video saved when something fails, so problems are quick to understand. Genuinely flaky tests are quarantined and retried so they do not hide real failures.

In short

I would use WebDriverIO with Appium in TypeScript for the cross platform user journeys, and lean on the native tools for the developers' lower level tests. I would build everything around Page Objects and a single map of element locations, so tests are written once and run everywhere. I would run them on simulators and emulators locally, on a device cloud for breadth, and gate every pull request with the fast web tests. The project here already does the core of this and runs successfully today.

Issues and Improvements Tracker

This is the full list of what I found while exploring automationexercise.com , treated as the reference for the mobile app. I grouped the issues by area and gave each one an ID so it can be followed through to a fix.

How I am using the labels:

Severity: Critical (a blocker, or affects integrity), High (a major function is broken), Medium (works only partly, or hurts usability), Low (minor or cosmetic). **Type:** Bug, Validation, UX, Enhancement, Stability.

Platform: Both, Mobile, Desktop.

Summary

ID	Area	Title	Platform	Severity	Type	Status
PAY-01	Checkout and payment	Payment accepts any input and confirms the order	Both	Critical	Validation	Open
AUTH-01	Account and auth	A badly formed email (paulo@test) is accepted at signup	Both	High	Validation	Open
AUTH-02	Account and auth	Registration accepts any value in any field	Both	High	Validation	Open
AUTH-04	Account and auth	"Delete Account" runs with no confirmation	Both	High	UX	Open
CART-01	Cart	You cannot change the quantity in the cart	Both	High	Bug	Open
CART-03	Cart	No overall cart total is shown on the cart screen	Both	High	Bug	Open
ATC-02	Add to cart	On mobile, a second tap is needed to add a product	Mobile	High	Bug	Open
BRW-01	Browsing and search	A category filter shows an unrelated product (Kids, Dress shows a non dress)	Both	Medium	Bug	Open
BRW-02	Browsing and search	Prices have no currency or formatting (Rs . 1500)	Both	Medium	UX	Open
BRW-03	Browsing and search	The Enter key does not run a search (desktop)	Desktop	Medium	UX	Open
AUTH-03	Account and auth	On mobile, the duplicate email error scrolls out of view	Mobile	Medium	UX	Open

ID	Area	Title	Platform	Severity	Type	Status
AUTH-05	Account and auth	There is no profile area to view or update account details	Both	Medium	Enhancement	Open
ATC-01	Add to cart	The "Added" popup does not block the page behind it	Both	Medium	UX	Open
ATC-03	Add to cart	On mobile, the "Added" popup appears in the wrong place	Mobile	Medium	UX	Open
CART-05	Cart	Tapping a product image in the cart reloads the cart instead of opening the product	Both	Medium	Bug	Open
CART-06	Cart	There is no maximum quantity, so unlimited units can be added	Both	Medium	Validation	Open
STAB-01	Stability	A "heavy load (queue full)" page and server errors after repeated use	Both	Medium	Stability	Open
FORM-01	Forms (across the app)	Empty required fields show a message but no highlight	Both	Low	UX	Open
CART-02	Cart	Removing a cart item has no confirmation	Both	Low	UX	Open
CART-04	Cart	The line totals (price times quantity) have no currency formatting	Both	Low	UX	Open
BRW-04	Browsing and search	The search button is not aligned with the search bar	Both	Low	UX	Open

Totals: 1 Critical, 6 High, 9 Medium, 5 Low. That is 21 issues.

A. Browsing and search

BRW-01. A category filter shows an unrelated product

Severity: Medium. **Type:** Bug. **Platform:** Both. Steps: go to Filter, then Category, then Kids, then Dress. Expected: only dresses appear. Actual: "Sleeves Top and Short - Blue & Pink", which is not a dress, is shown. Improvement: fix which products belong to which category, and add a check so products only appear under categories that match. This is also worth an automated test on filtered results.

BRW-02. Prices have no currency or formatting

Severity: Medium. **Type:** UX. **Platform:** Both. Steps: look at the Home or Products screen. Expected: a clearly formatted price with a currency, for example ₹1,500.00 . Actual: plain values like Rs. 1500 ,

with an unclear currency and no grouping or decimals. Improvement: show prices formatted for the user's region, with the right currency and proper number formatting. This matters for trust, and even more on a global mobile app.

BRW-03. The Enter key does not run a search (desktop)

Severity: Medium. **Type:** UX. **Platform:** Desktop. Steps: on Products, type "T-Shirt" in the search box and press Enter. Expected: the search runs when I press Enter. Actual: nothing happens. Only clicking the search button works. Improvement: make the search run on Enter. On mobile, the keyboard's search or go key should also trigger it.

BRW-04. The search button is not aligned with the bar

Severity: Low. **Type:** UX. **Platform:** Both. Expected: the search button lines up with the input. Actual: the button is offset from the search bar. Improvement: fix the alignment so the button sits neatly with the input.

B. Account and authentication

AUTH-01. A badly formed email is accepted at signup

Severity: High. **Type:** Validation. **Platform:** Both. Steps: go to Signup and Login, then New User Signup, enter paulo@test , and tap Signup. Expected: an email with no real domain is rejected, with a clear message. Actual: the account is created. Improvement: check the email format properly, both in the app and on the server, and ideally confirm it by sending a verification email. This keeps out junk and unreachable accounts.

AUTH-02. Registration accepts any value in any field

Severity: High. **Type:** Validation. **Platform:** Both. Steps: fill in the registration form with random values for Password, Address, Phone, Zip, and so on. Expected: each field is checked appropriately (password rules, numbers for phone and zip, sensible lengths). Actual: any input is accepted. Improvement: add proper rules per field, enforced on the server too. This protects data quality and the later shipping and payment steps.

AUTH-03. On mobile, the duplicate email error scrolls out of view

Severity: Medium. **Type:** UX. **Platform:** Mobile. Steps: on a phone, go to New User Signup, enter an email that already exists, and tap Signup. Expected: the message "Email Address already exist!" is visible. Actual: the page jumps to the top, so the error, which sits lower, is off screen. The user has to scroll to find it. Improvement: scroll to the error or show it next to the field. This also relates to Discovery Q1.

AUTH-04. "Delete Account" runs with no confirmation

Severity: High. **Type:** UX (an action that cannot be undone). **Platform:** Both. Steps: while logged in, click Delete Account. Expected: a confirmation step before something permanent. Actual: the account is

deleted right away. Improvement: add a confirmation step, and ideally ask for the password again, before anything permanent.

AUTH-05. There is no profile area

Severity: Medium. **Type:** Enhancement. **Platform:** Both. Expected: a place to view and update profile details such as name, address, and password. Actual: there is no profile or account management area. Improvement: add a profile screen for viewing and editing the user's details.

C. Forms and validation (across the app)

FORM-01. Empty required fields are not highlighted

Severity: Low. **Type:** UX. **Platform:** Both. Steps: submit any form with a required field left empty. Expected: the empty field is highlighted, for example with a red border, alongside a message. Actual: only a message appears, with no highlight, so it is not obvious which field needs attention. Improvement: apply a clear error style (red border, helper text, and focus) to empty or invalid required fields everywhere in the app.

D. Adding to the cart

ATC-02. On mobile, a second tap is needed to add a product

Severity: High. **Type:** Bug. **Platform:** Mobile. Steps: on a phone, go to Products and tap Add to Cart on a product. Expected: one tap adds the product. Actual: an orange slider appears first, and I have to tap Add to Cart again to actually add it. Improvement: remove the in between hover style state on touch devices so the first tap adds the product. This is friction on the most important action.

ATC-01. The "Added" popup does not block the page behind it

Severity: Medium. **Type:** UX. **Platform:** Both. Steps: add a product to the cart. Expected: the confirmation either blocks the page until dismissed, or is clearly a non blocking notice by design. Actual: the page behind the popup can still be scrolled and tapped without closing it. Improvement: decide whether this is a blocking popup or a light notification, and make it consistent. Either block the page behind it, or replace it with a small toast plus a cart badge.

ATC-03. On mobile, the "Added" popup appears in the wrong place

Severity: Medium. **Type:** UX. **Platform:** Mobile. Steps: on a phone, add a product to the cart. Expected: a clearly visible, centred confirmation. Actual: the popup appears toward the top of the screen rather than centred. Improvement: fix the popup's position so it is centred and clearly visible on phone screens.

E. Cart

CART-01. You cannot change the quantity in the cart

Severity: High. **Type:** Bug. **Platform:** Both. **Steps:** add a product, open the cart, and try to change the quantity. **Expected:** the quantity can be edited in the cart. **Actual:** the quantity cannot be changed in the cart. **Improvement:** make the quantity editable, with the total updating live. This is core cart behaviour.

CART-03. No overall cart total is shown

Severity: High. **Type:** Bug. **Platform:** Both. **Steps:** add several products and open the cart. **Expected:** a clear overall total (the sum of all the line totals). **Actual:** each line shows its own total (price times quantity), but there is no overall total. The user has to go to checkout to find out what they will pay. **Improvement:** show a running subtotal and overall total on the cart screen. This is separate from CART-04, which is about the formatting of the existing line totals.

CART-05. Tapping a product image in the cart reloads the cart

Severity: Medium. **Type:** Bug. **Platform:** Both. **Steps:** in the cart, tap a product's image. **Expected:** it opens that product's page. **Actual:** the page reloads and the cart opens again, instead of the product. **Improvement:** link the cart product image and title to the correct product page.

CART-06. There is no maximum quantity

Severity: Medium. **Type:** Validation. **Platform:** Both. **Steps:** add the same product many times, or set a very large quantity. **Expected:** a sensible limit, based on stock or a per order cap. **Actual:** unlimited units can be added. **Improvement:** set a maximum quantity tied to stock or business rules, with a clear message when the limit is reached.

CART-02. Removing a cart item has no confirmation

Severity: Low. **Type:** UX. **Platform:** Both. **Steps:** in the cart, tap the "x" on an item. **Expected:** a confirmation, or an undo option. **Actual:** the item is removed instantly with no confirmation. **Improvement:** add an undo option (my preference) or a confirmation for removals.

CART-04. The line totals have no currency formatting

Severity: Low. **Type:** UX. **Platform:** Both. **Steps:** add a product with a quantity above 1, open the cart, and look at the line total (price times quantity). **Expected:** a formatted amount, for example 6,300.00 with a currency. **Actual:** the line total is a plain number like 6300. **Improvement:** format every amount with the right currency and grouping. Same root cause as BRW-02, and separate from CART-03, which is the missing overall total.

F. Checkout and payment

PAY-01. Payment accepts any input and confirms the order

Severity: Critical. **Type:** Validation. **Platform:** Both. **Steps:** reach the payment screen, type 1 into every field, and submit. **Expected:** the card number, expiry, and security code are checked, and invalid

details are rejected. Actual: the order is confirmed with clearly invalid details. Improvement: check the payment fields properly (format, length, expiry, security code) and connect a real or sandbox payment provider. This is the most important issue, since it directly affects whether orders and payments are real. On mobile, also guard against the same payment being submitted twice on a shaky connection.

G. Stability and performance

STAB-01. A "heavy load (queue full)" page after repeated use

Severity: Medium. **Type:** Stability. **Platform:** Both. Steps: browse the site repeatedly during a session. Expected: the site stays available, or handles being overloaded gracefully with a clear message. Actual: a "This website is under heavy load (queue full)" page appears, along with server and network errors in the browser tools. Improvement: I captured the console and network output as evidence. This is most likely the public demo site hitting its own capacity limit, but the way it is shown to users and the errors behind it are still worth noting. On a real app, I would want graceful handling, retries, and a clear message.

How these connect to the assignment

- For the exploratory report, I promoted the most important issues (such as PAY-01, ATC-02, CART-01, AUTH-03, and AUTH-04) into the main "issues" section with full steps and impact, and pointed to this tracker for the complete list.
- For Discovery Q1 (an email that already exists), AUTH-01 and AUTH-03 are the direct evidence.
- For Discovery Q2 (the cart after a refresh), the behaviour is confirmed by the automated test (the cart persists), with a manual check to match.
- For Discovery Q3 (something confusing while browsing), BRW-02 (unclear prices), BRW-01 (a wrong filter result), and BRW-03 (Enter does not search) are all strong candidates.